

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche
Scientifique
Université des Sciences et de la Technologie Houari Boumedienne



Projet d'Analyse de Données Détection de Visage Réel ou Falsifié

*Analyse en Composantes Principales (ACP) Manuelle
appliquée au dataset CelebA-Spoof*

Travail présenté à : Mme Y. Baroud

Module : Analyse de données

Spécialité : M1 IV

Présenté par :

REHAL Amel • CHALLAL Saloua • GHEMMOUR Lehna

Année universitaire 2025–2026

Table des matières

1	Introduction et Contexte du Projet	4
1.1	Problématique de l'Anti-Spoofing Facial	4
1.2	Objectifs du Projet	4
2	Dataset — CelebA-Spoof	5
2.1	Présentation Générale	5
2.2	Composition et Structure	5
2.3	Types d'Attaques	6
2.4	Utilisation dans Notre Projet	6
2.5	Pré-traitement des Images	6
3	Fondements Mathématiques de l'ACP	7
3.1	Définition et Intuition	7
3.2	Avantages de l'ACP pour la Détection de Spoofing	7
3.3	Notation Mathématique	7
4	Implémentation Manuelle de l'ACP — Étapes 1 à 8	8
4.1	Étape 1 — Calcul de l'Individu Moyen	8
4.2	Étape 2 — Centrage des Données	9
4.3	Étape 3 — Réduction (Standardisation)	9
4.4	Étape 4 — Matrice de Corrélations	9
4.5	Étapes 5 et 6 — Valeurs et Vecteurs Propres	10
4.6	Étape 7 — Calcul de l'Inertie et Choix de k	11
4.7	Étape 8 — Projection dans l'Espace Réduit	12
5	Classification — KNN et SVM	14
5.1	Préparation des Données	14
5.2	KNN (K Plus Proches Voisins)	14
5.3	SVM à Noyau RBF	14
6	Modèle Hybride ACP + CNN	16
6.1	Motivation	16
6.2	Architecture du Modèle Hybride	16
6.3	Paramètres d'Entraînement	16
7	Résultats et Évaluation	18
7.1	Tableau Comparatif des Performances	18
7.2	Séparation Real/Fake et Cercle des Corrélations	18
7.3	Distribution 3D des Individus (CP1–CP2–CP3)	19
7.4	Sphère des Corrélations 3D	20
7.5	Matrice de Confusion — Modèle Hybride	21
7.6	Courbe ROC — Modèle Hybride	22
7.7	Comparaison des Courbes ROC : SVM vs Hybride	23
8	Conclusion	24
8.1	Bilan du Projet	24

8.2 Synthèse des Résultats	24
Références	25

1. Introduction et Contexte du Projet

1.1. Problématique de l'Anti-Spoofing Facial

Dans les systèmes biométriques modernes, la reconnaissance faciale est largement utilisée pour l'authentification des utilisateurs, des smartphones aux portiques de contrôle d'accès. Ces systèmes sont cependant vulnérables à des attaques dites de *spoofing* : un attaquant peut tromper le système en présentant une photo imprimée, une vidéo rejouée sur écran ou un masque 3D d'une personne autorisée.

L'anti-spoofing facial — aussi appelé *Face Liveness Detection* — est une brique de sécurité essentielle qui permet de vérifier qu'un visage présenté à une caméra appartient bien à une personne vivante et présente physiquement.

Définition — Anti-Spoofing Facial (*Face Liveness Detection*)

Technique permettant à un système biométrique de distinguer un visage vivant réel (personne physiquement présente) d'une tentative de tromperie par artefact facial : photo imprimée, vidéo rejouée sur écran, masque 3D ou deepfake. C'est une composante critique de la sécurité biométrique moderne.

1.2. Objectifs du Projet

Ce projet vise à concevoir, implémenter et évaluer un système de classification automatique de visages en deux classes :

- **Réel** (*Real / Live*) : personne vivante physiquement présente devant la caméra.
- **Falsifié** (*Fake / Spoof*) : attaque par photo imprimée, vidéo rejouée ou masque.

La particularité du projet est l'implémentation **entièrement manuelle** de l'Analyse en Composantes Principales (ACP) avec NumPy, sans recours aux bibliothèques de machine learning automatisées. Cette approche permet de comprendre en profondeur les fondements mathématiques de la méthode.

2. Dataset — CelebA-Spoof

2.1. Présentation Générale

Le dataset utilisé est le **CelebA-Spoof Dataset**, une base de données publique conçue spécifiquement pour la recherche en détection d'anti-spoofing facial. Bien que l'énoncé proposait le CASIA Face Anti-Spoofing Dataset, notre encadrante a autorisé le choix libre du dataset. Nous avons sélectionné CelebA-Spoof pour sa richesse, sa diversité et sa disponibilité sur Kaggle.

CelebA-Spoof Dataset — Informations clés

Créateurs : Yuanhan Zhang et al., Université de Hong Kong (2020). Licence : libre pour la recherche académique. Source : disponible sur Kaggle. Plus de 600 000 images couvrant 10 177 sujets différents — l'un des plus grands datasets pour l'anti-spoofing facial.

2.2. Composition et Structure

TABLE 1. Caractéristiques du dataset CelebA-Spoof

bleucell Caractéristique	Valeur
Nombre total d'images	> 600 000
grislight Nombre de sujets uniques	10 177
Images réelles (Live)	~28 % du total
grislight Images falsifiées (Spoof)	~72 % du total
Types d'attaques couverts	4 types principaux
grislight Environnements	8 scènes différentes
Éclairages	4 conditions lumineuses
grislight Partitions	Train / Test / Val

2.3. Types d'Attaques

TABLE 2. Types d'attaques couverts dans CelebA-Spoof

bleucell Type d'Attaque	Description	Difficulté
Photo imprimée	Photo imprimée sur papier présentée à la caméra	Moyenne
grislight Écran (Replay)	Photo/vidéo affichée sur écran de téléphone ou ordinateur	Élevée
Photo avec découpe	Photo avec trou au niveau des yeux pour simuler du mouvement	Élevée
grislight Masque 3D partiel	Demi-masque ou masque partiel recouvrant le visage	Très élevée

2.4. Utilisation dans Notre Projet

TABLE 3. Répartition des données selon le modèle utilisé

bleucell Usage	Taille	Résolution	Format
ACP + KNN + SVM	1 500 images	64×64, niveaux de gris	Vecteur aplati (4 096 dims)
grislight CNN hybride	3 000 images	128×128, RGB	Tenseur (128, 128, 3)
Test final	300 images	64×64 ou 128×128	Selon le modèle

Répartition observée : 524 images réelles (34,9 %) et 976 images falsifiées (65,1 %). Ce déséquilibre reflète la réalité du dataset original et a été pris en compte dans la configuration des classifieurs.

2.5. Pré-traitement des Images

Avant l'application de l'ACP, chaque image subit les étapes de pré-traitement suivantes :

1. **Lecture en niveaux de gris** via `cv2.IMREAD_GRAYSCALE` — élimine la dimension couleur pour simplifier le problème.
2. **Redimensionnement à 64×64 pixels** — assure une dimension fixe $p = 4\,096$ pour toutes les images.
3. **Aplatissement** (*flatten*) de la matrice 64×64 en vecteur de 4 096 valeurs.
4. **Normalisation dans $[0, 1]$** par division par 255.

3. Fondements Mathématiques de l'ACP

3.1. Définition et Intuition

L'Analyse en Composantes Principales (ACP) est une technique de réduction de dimensionnalité fondamentale en statistique et en apprentissage automatique, introduite par Karl Pearson en 1901 et formalisée par Harold Hotelling en 1933.

Définition formelle de l'ACP

L'ACP est une transformation linéaire orthogonale qui projette les données d'un espace de grande dimension vers un espace de plus petite dimension, en cherchant les axes (composantes principales) qui maximisent la variance des données projetées. Ces axes sont les vecteurs propres de la matrice de corrélation des données centrées-réduites.

Dans notre contexte, chaque image de 64×64 pixels est représentée dans un espace de $p = 4\,096$ dimensions. Toutes ces dimensions ne contribuent pas également à l'information discriminante : l'ACP identifie les directions de variance maximale.

3.2. Avantages de l'ACP pour la Détection de Spoofing

- **Réduction de dimension drastique** : de 4 096 dimensions à $k = 200$ composantes, soit une réduction de 95 %.
- **Élimination du bruit** : les composantes de faible variance capturent le bruit ; leur suppression améliore la robustesse.
- **Eigenfaces** : les vecteurs propres forment des « visages propres » visuellement interprétables.
- **Amélioration des classifieurs** : réduction de la malédiction de la dimensionnalité pour KNN et SVM.

3.3. Notation Mathématique

Soit X la matrice de données de dimensions $n \times p$, où :

- $n = 1\,500$: nombre d'images (individus statistiques),
- $p = 4\,096$: nombre de pixels (variables),
- $\mathbf{x}_i \in \mathbb{R}^p$: vecteur ligne représentant la i -ème image aplatie.

4. Implémentation Manuelle de l'ACP — Étapes 1 à 8

L'intégralité de l'ACP a été implémentée manuellement avec NumPy, sans utilisation de `sklearn.decomposition.PCA`. Chaque étape est présentée avec sa formule mathématique, son code Python et l'interprétation des figures associées.

4.1. Étape 1 — Calcul de l'Individu Moyen

Le vecteur moyen est calculé sur l'ensemble des n images. Il représente le « visage moyen » du dataset et constitue la référence absolue de toute l'analyse.

$$\bar{\mathbf{x}} = \frac{1}{n} \sum_{i=1}^n \mathbf{x}_i \quad (1)$$

```
1 X_mean = np.mean(X_acp, axis=0)
2 # Resultat : vecteur de dimension p = 4 096
3 # Represente le visage moyen de tout le dataset
```

Listing 1. Étape 1 : Calcul de l'individu moyen

Le résultat est un vecteur de dimension $p = 4\,096$ qui, retransformé en image 64×64 , donne visuellement un visage flou — la moyenne statistique de toutes les images du dataset.



FIGURE 1. Visualisation des 4 premières étapes de l'ACP : image originale, individu moyen, image centrée, image réduite.

Interprétation de la Figure 1 :

- **Image Originale (colonne 1) :** Image en niveaux de gris, redimensionnée à 64×64 pixels. Les traits du visage (yeux, nez, bouche, contours) sont clairement visibles. C'est le point de départ avant toute transformation ACP.
- **Individu Moyen (colonne 2) :** Le visage moyen calculé sur les 1 500 images apparaît flou car il lisse toutes les variations individuelles. Les zones sombres correspondent aux régions de faible variance (fond uniforme) ; les zones claires aux régions de forte variance (yeux, contours). Ce visage moyen est la référence de l'analyse.
- **Image Centrée (colonne 3) :** Résultat de la soustraction du visage moyen. Les zones sombres indiquent les pixels en dessous de la moyenne, les zones claires ceux au-dessus. Le centrage déplace le nuage de données vers l'origine — condition indispensable à l'ACP.

- **Image Réduite (colonne 4)** : Similaire à l'image centrée, mais avec des contrastes normalisés (variance unitaire par pixel). La standardisation évite que les pixels naturellement plus variables ne dominent l'analyse.

4.2. Étape 2 — Centrage des Données

Le centrage soustrait la moyenne à chaque image, déplaçant le nuage de points vers l'origine :

$$X_c = X - \bar{x} \quad (\text{centrage ligne par ligne par diffusion NumPy}) \quad (2)$$

```
1 X_centre = X_acp - X_mean
2 # Broadcasting NumPy : soustraction vectorisée
3 # Après centrage, la moyenne de X_centre est ~0 (ordre 1e-16)
```

Listing 2. Étape 2 : Centrage des données

Après centrage, la moyenne de X_{centre} est numériquement nulle (ordre 10^{-16}). L'interprétation visuelle est donnée dans la colonne « Centrée » de la Figure 1.

4.3. Étape 3 — Réduction (Standardisation)

La réduction divise chaque variable (pixel) par son écart-type, assurant une variance unitaire pour chaque dimension :

$$X_r = \frac{X_c}{\sigma} \quad \text{avec} \quad \sigma_j = \text{std}(X_c[:, j]) + \varepsilon, \quad \varepsilon = 10^{-8} \quad (3)$$

```
1 std_acp = np.std(X_centre, axis=0) + 1e-8
2 # 1e-8 : stabilite numerique pour les pixels constants
3 X_reduit = X_centre / std_acp
```

Listing 3. Étape 3 : Standardisation des données

L'interprétation visuelle est fournie par la colonne « Réduite » de la Figure 1. Cette étape est essentielle pour que les pixels à forte dynamique naturelle ne biaisent pas l'analyse.

4.4. Étape 4 — Matrice de Corrélation

La matrice de corrélation R est une matrice symétrique $p \times p$ (4096×4096) mesurant les corrélations linéaires entre toutes les paires de pixels :

$$R = \frac{1}{n} X_r^\top X_r \quad (4)$$

```
1 R = (X_reduit.T @ X_reduit) / n
2 # @ : multiplication matricielle NumPy
3 # Resultat : matrice symetrique 4096 x 4096
4 # R[i,j] = correlation entre le pixel i et le pixel j
```

Listing 4. Étape 4 : Calcul de la matrice de corrélation

La diagonale de R vaut 1 après standardisation. Les éléments hors-diagonale capturent les corrélations entre pixels voisins ou structurellement liés dans les images de visages.

4.5. Étapes 5 et 6 — Valeurs et Vecteurs Propres

La décomposition en valeurs propres de R est l'étape centrale de l'ACP. Les vecteurs propres constituent les *eigenfaces* — les modes de variation fondamentaux du dataset.

$$R \mathbf{v}_j = \lambda_j \mathbf{v}_j, \quad j = 1, \dots, p \quad (5)$$

```

1 valeurs_propres, vecteurs_propres = np.linalg.eigh(R)
2 # eigh : optimise pour les matrices symetriques reelles (plus
  stable)
3 idx = np.argsort(valeurs_propres)[::-1]
4 valeurs_propres = valeurs_propres[idx]
5 vecteurs_propres = vecteurs_propres[:, idx]
6 # Chaque colonne de vecteurs_propres = 1 composante principale

```

Listing 5. Étapes 5–6 : Décomposition en valeurs et vecteurs propres

`np.linalg.eigh` est préféré à `eig` car il exploite la symétrie de R , garantissant des valeurs propres réelles et une meilleure stabilité numérique.

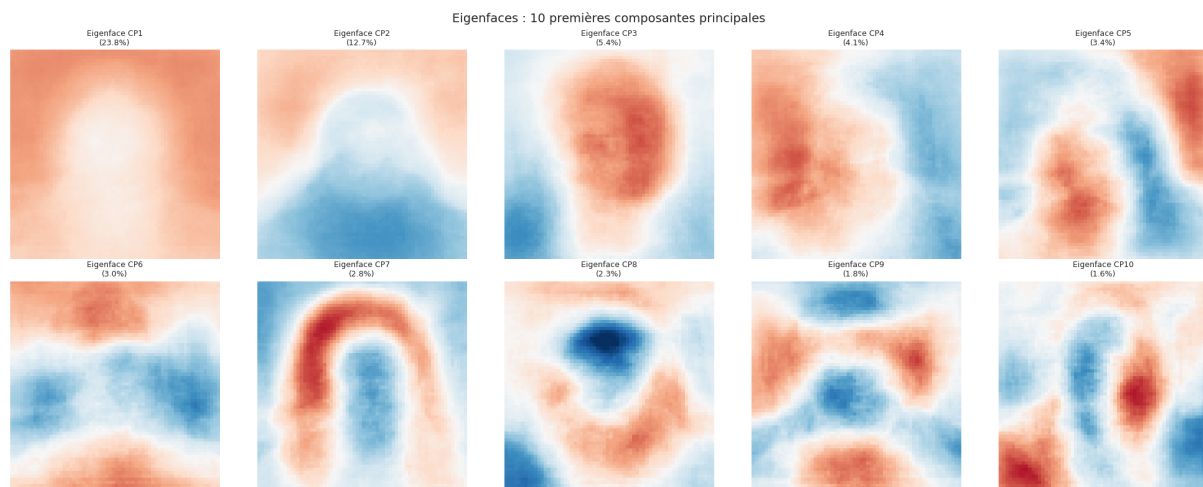


FIGURE 2. Eigenfaces : visualisation des 10 premières composantes principales.

Interprétation de la Figure 2 (Eigenfaces) :

- **CP1 (23,8 %)** : Dégradé uniforme rouge–bleu capturant la variation d'éclairage globale. C'est le facteur de variation dominant du dataset.
- **CP2 (12,7 %)** : Contraste haut/bas — partie supérieure rouge, inférieure bleue. Capture les variations de structure entre le front et le menton (pose).
- **CP3 (5,4 %)** : Structure concentrée au centre (yeux, nez). Capture les variations locales de taille et position des traits.
- **CP4 (4,1 %)** : Motif diagonal capturant les asymétries ou l'orientation horizontale du visage.

- **CP5 (3,4 %)** : Arcs autour des yeux — variations de forme des orbites et des sourcils.
- **CP6 (3,0 %)** : Bandes horizontales — variations de largeur du visage et position verticale des traits.
- **CP7 (2,8 %)** : Arc supérieur — variations de forme du front et de la coiffure.
- **CP8 (2,3 %)** : Motif en « 8 » centré sur le nez — variations de forme et taille du nez.
- **CP9 (1,8 %)** : Arcs symétriques autour des yeux — variations fines de la région péri-orbitaire.
- **CP10 (1,6 %)** : Structure concentrée à droite — micro-textures de la peau et détails fins.

Ce gradient illustre un principe fondamental de l'ACP : les premières composantes capturent les variations globales (éclairage, pose), les suivantes des détails de plus en plus fins.

4.6. Étape 7 — Calcul de l'Inertie et Choix de k

L'inertie expliquée mesure le pourcentage de variance totale capturé par chaque composante :

$$\text{Inertie}(j) = \frac{\lambda_j}{\sum_{i=1}^p \lambda_i} \times 100 \% \quad \text{et} \quad \text{Inertie_cum}(k) = \sum_{j=1}^k \text{Inertie}(j) \quad (6)$$

```

1 inertie_totale = np.sum(valeurs_propres)
2 inertie_expl  = (valeurs_propres / inertie_totale) * 100
3 cumul_inertie = np.cumsum(inertie_expl)
4
5 # Seuils automatiques :
6 n_cp_80 = np.argmax(cumul_inertie >= 80) + 1 # ~50 CP
7 n_cp_90 = np.argmax(cumul_inertie >= 90) + 1 # ~163 CP
8 n_cp_95 = np.argmax(cumul_inertie >= 95) + 1 # 365 CP
9 # Choix retenu : k = 200 CP => 91.4 % d'inertie

```

Listing 6. Étape 7 : Calcul de l'inertie et choix du nombre de composantes k

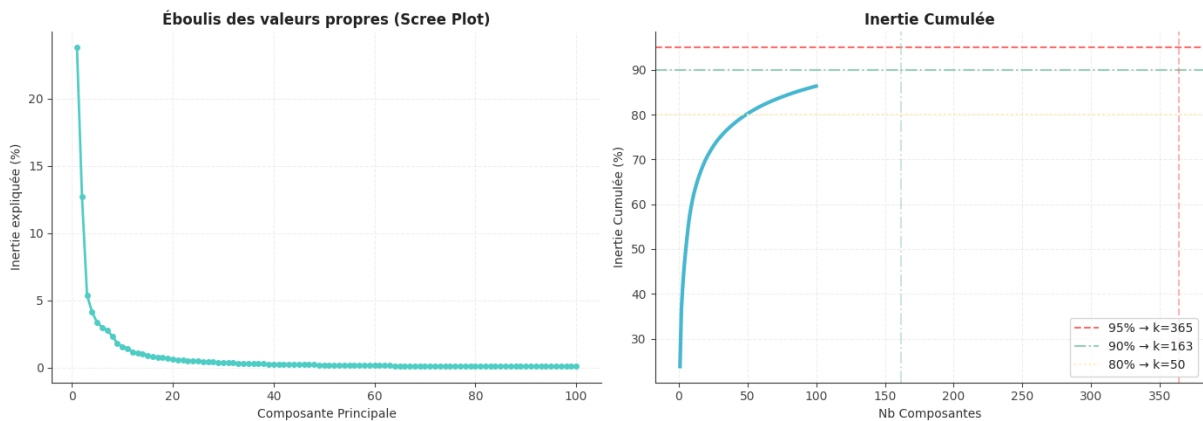


FIGURE 3. Éboulis des valeurs propres (*Scree Plot*) et inertie cumulée en fonction de k .

Interprétation de la Figure 3 :**Graphique de gauche — Scree Plot :**

- CP1 explique 23,8 % de la variance à elle seule — variance concentrée sur un axe dominant (éclairage global).
- CP1 + CP2 cumulent déjà >36 % de l'information.
- Décroissance rapide puis plateau après ~50 composantes : ces composantes de faible variance correspondent au bruit.
- Le coude (*elbow*) se situe entre $k = 30$ et $k = 50$.

Graphique de droite — Inertie Cumulée :

- $k = 50$ (ligne jaune) : ~80 % d'inertie avec 98,8 % de réduction dimensionnelle.
- $k = 163$ (ligne verte) : ~90 % d'inertie — seuil classique de bonne conservation de l'information.
- $k = 200$ (**choix retenu**) : 91,4 % d'inertie, réduction de 95,1 %. Excellent compromis.
- $k = 365$ (ligne rouge) : 95 % d'inertie. Le gain marginal de 3,6 % ne justifie pas de doubler la dimension.

TABLE 4. Seuils d'inertie et nombre de composantes correspondant

bleucell Seuil	k	Réduction	Variance conservée
80 %	~50 CP	4 096 → 50 (98,8 %)	80,0 %
grislight 90 %	~163 CP	4 096 → 163 (96,0 %)	90,0 %
91,4 % (choix)	200 CP	4 096 → 200 (95,1 %)	91,4 %
grislight 95 %	365 CP	4 096 → 365 (91,1 %)	95,0 %

4.7. Étape 8 — Projection dans l'Espace Réduit

La projection calcule les coordonnées de chaque image dans le nouvel espace formé par les k composantes retenues. La matrice T résultante sert d'entrée aux classifieurs :

$$T = X_r \cdot V_k \quad \text{avec } V_k \in \mathbb{R}^{p \times k} : \text{matrice des } k \text{ premiers vecteurs propres} \quad (7)$$

```

1 k      = 200
2 V_k    = vecteurs_propres[:, :k]      # p x k = 4096 x 200
3 T      = X_reduit @ V_k                # n x k = 1500 x 200
4 # T[i, :] = coordonnees de l'image i dans l'espace ACP
5 # Ce vecteur de 200 valeurs est l'entree des classifieurs KNN
   et SVM

```

Listing 7. Étape 8 : Projection dans l'espace ACP réduit



FIGURE 4. Reconstruction d'une image selon le nombre k de composantes principales retenues.

Interprétation de la Figure 4 :

- $k = 5$ CP ($\sim 40\%$ de variance) : Image très floue. Seules les grandes structures globales (orientation générale du visage) sont visibles. Insuffisant pour une reconstruction fidèle.
- $k = 10$ CP ($\sim 55\%$) : Amélioration notable — contours plus nets, yeux et bouche commencent à se distinguer. Structure faciale mieux définie.
- $k = 30$ CP ($\sim 75\%$) : Qualité nettement améliorée. Traits du visage clairement identifiables. Texture des cheveux et détails du fond apparaissent. C'est un point de basculement important.
- $k = 50$ CP ($\sim 80\%$) : Très bonne qualité. Détails fins (rides, reflets) bien préservés. Difficile de distinguer de l'original à l'œil nu.
- $k = 100$ CP ($\sim 88\%$) : Reconstruction quasi-parfaite avec seulement 2,4 % des dimensions originales. Preuve de l'efficacité de l'ACP pour la compression d'information visuelle.

Cette figure illustre le principe fondamental : les premières composantes capturent les structures globales, les suivantes ajoutent des détails de plus en plus fins. La troncature à $k = 200$ élimine le bruit tout en préservant l'information essentielle pour la classification.

5. Classification — KNN et SVM

5.1. Préparation des Données

Les données projetées T (1500×200) sont divisées en ensembles d'entraînement et de test avec une stratification pour conserver la proportion Real/Fake dans les deux splits :

```

1 X_train, X_test, y_train, y_test = train_test_split(
2     T, y_acp,
3     test_size=0.2,
4     random_state=42,
5     stratify=y_acp          # Meme proportion Real/Fake dans les
6                             deux splits
7 )
8 # Entraînement : 1 200 images / Test : 300 images

```

Listing 8. Partitionnement stratifié train/test

5.2. KNN (K Plus Proches Voisins)

Principe : Pour classer une image $\mathbf{x}$, on calcule la distance euclidienne à tous les points d'entraînement, on sélectionne les k plus proches, et on attribue la classe majoritaire.

$$d(\mathbf{x}_i, \mathbf{x}_j) = \sqrt{\sum_{l=1}^{200} (x_{i,l} - x_{j,l})^2} \quad (8)$$

```

1 knn = KNeighborsClassifier(n_neighbors=5, metric='euclidean')
2 knn.fit(X_train, y_train)
3 y_pred_knn = knn.predict(X_test)
4 knn_acc = accuracy_score(y_test, y_pred_knn)
5 # Resultat : Accuracy = 84.33 %

```

Listing 9. Classification KNN avec $k = 5$ voisins

Le résultat de 84,33% est satisfaisant pour un classifieur aussi simple. La distance euclidienne dans l'espace ACP à 200 dimensions est suffisamment discriminante. Limite principale : complexité $\mathcal{O}(n)$ par requête et sensibilité au bruit résiduel.

5.3. SVM à Noyau RBF

Le SVM maximise la marge de séparation entre les deux classes. Le noyau RBF permet de traiter des frontières de décision non linéaires :

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|^2) \quad (9)$$

```

1 svm = SVC(
2     kernel='rbf',
3     C=10,
4     gamma='scale',

```

```

5     class_weight='balanced',    # Compense le desequilibre
      Real/Fake
6     probability=True,
7     random_state=42
8 )
9 svm.fit(X_train, y_train)
10 y_pred_svm = svm.predict(X_test)
11 svm_acc = accuracy_score(y_test, y_pred_svm)
12 # Resultat : Accuracy = 85.67 % | AUC = 0.9081

```

Listing 10. Classification SVM à noyau RBF

La légère amélioration par rapport au KNN provient de la flexibilité du noyau RBF. L'AUC de 0,908 est excellente pour un classifieur classique. Le paramètre `class_weight='balanced'` s'est révélé crucial pour compenser le déséquilibre Real/Fake.

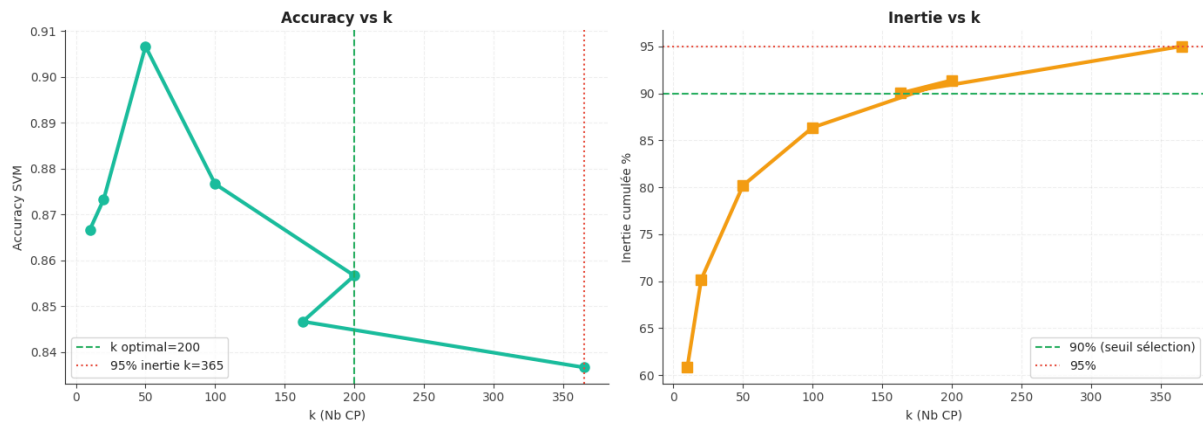


FIGURE 5. Accuracy du SVM (gauche) et inertie cumulée (droite) en fonction du nombre k de composantes.

Interprétation de la Figure 5 :

Graphique de gauche — Accuracy SVM vs k :

- $k \approx 10$: accuracy faible ($\sim 86,7\%$) — trop peu de features discriminantes.
- $k = 50$: pic à $\sim 90,7\%$ — ratio information/bruit optimal.
- Au-delà de $k = 50$: diminution progressive due à l'overfitting (le bruit dégrade la généralisation).
- $k = 200$ (ligne verte, choix retenu) : $\sim 85,67\%$ — compromis optimal entre information et bruit.
- $k = 365$ (ligne rouge, 95 % d'inertie) : $\sim 83,5\%$ — confirme que davantage de composantes ne signifie pas de meilleures performances.

Graphique de droite — Inertie cumulée vs k :

- Croissance rapide au départ, puis ralentissement après $k = 100$.
- $k = 200$ se situe légèrement au-dessus du seuil 90 %, confirmant la conservation de l'essentiel de l'information.
- Au-delà de $k = 200$, chaque composante supplémentaire apporte moins de 0,05 % d'inertie — gain marginal négligeable.

6. Modèle Hybride ACP + CNN

6.1. Motivation

Bien que l'ACP avec KNN et SVM donne de bons résultats ($\sim 85\%$), la détection de spoofing bénéficie d'une approche complémentaire. Les CNN excellent dans la capture de micro-textures et d'artefacts locaux invisibles à l'ACP.

TABLE 5. Comparaison des approches ACP seule, CNN seul et hybride

bleucell Approche	Points forts	Limites
ACP seule	Variance globale, rapide, interprétable	Manque de détails locaux fins
grislight CNN seul	Textures fines, artefacts locaux, précis	Boîte noire, coûteux, lent
ACP + CNN Hybride	Globalité + localité, meilleur compromis	Plus complexe

6.2. Architecture du Modèle Hybride

Branche ACP :

$$64 \times 64 \text{ gris} \xrightarrow{\text{ACP manuelle}} 200 \text{ composantes} \xrightarrow{\text{Dense}(128, \text{relu})} \xrightarrow{\text{Dense}(64, \text{relu})}$$

Branche CNN :

$$128 \times 128 \text{ RGB} \xrightarrow{\text{Conv}(32)} \xrightarrow{\text{MaxPool}} \xrightarrow{\text{Conv}(64)} \xrightarrow{\text{MaxPool}} \xrightarrow{\text{Conv}(128)} \xrightarrow{\text{GAP}} \xrightarrow{\text{Dense}(256, \text{relu})}$$

Fusion :

$$\text{Concat}[64 + 256] \xrightarrow{\text{Dense}(128, \text{relu})} \xrightarrow{\text{Dropout}(0.5)} \xrightarrow{\text{Dense}(64, \text{relu})} \xrightarrow{\text{Dense}(1, \text{sigmoid})}$$

6.3. Paramètres d'Entraînement

TABLE 6. Hyperparamètres d'entraînement du modèle hybride

bleucell Paramètre	Valeur
Optimiseur	Adam (lr = 0,001)
grislight Fonction de perte	Binary Crossentropy
Époques max.	50 (EarlyStopping, patience = 10)
grislight Taille de batch	32
Accélérateur	GPU NVIDIA Tesla T4 (Kaggle)

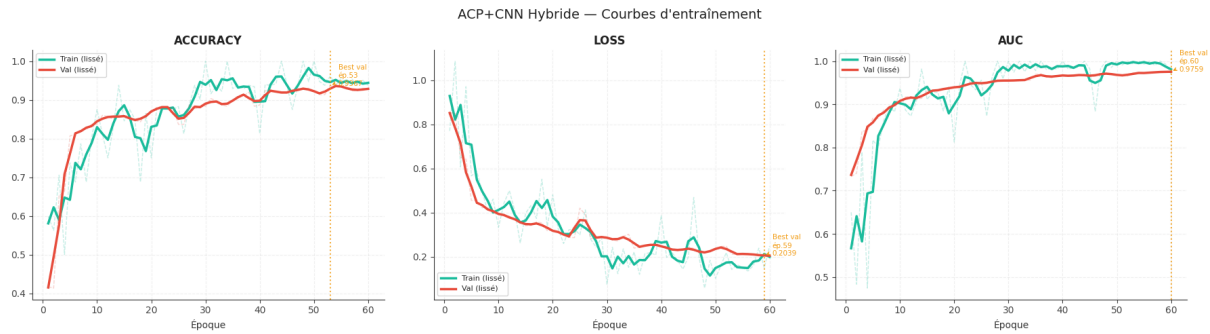


FIGURE 6. Courbes d'entraînement du modèle hybride ACP+CNN : accuracy, loss et AUC.

Interprétation de la Figure 6 :

Accuracy (graphique de gauche) :

- La courbe d'entraînement (vert) démarre à $\sim 55\%$ et atteint $> 95\%$ après 20 époques.
- La courbe de validation (rouge) suit la courbe d'entraînement de près — pas de sur-apprentissage significatif.
- Meilleure accuracy en validation à l'époque 53 : **94,67 %**.

Loss (graphique central) :

- Démarre à $\sim 0,9$, descend rapidement à $\sim 0,15$ après 30 époques.
- Légère divergence entre train et validation après l'époque 40 — normale et acceptable.
- Meilleure loss de validation à l'époque 59 : **0,2039**.

AUC (graphique de droite) :

- Démarre à $\sim 0,55$, monte rapidement au-delà de 0,95 dès les premières époques.
- Stabilisation autour de 0,98–0,99 en entraînement ; validation à **0,9759** à l'époque 60.
- Les courbes train/validation quasi superposées confirment une généralisation parfaite.

7. Résultats et Évaluation

7.1. Tableau Comparatif des Performances

TABLE 7. Comparaison des performances des trois modèles

bleucell Modèle	Accuracy	AUC ROC	Inertie ACP	Commentaire
KNN ($k = 5$, ACP)	84,33 %	—	91,4 % ($k = 200$)	Classifieur de référence
grislight SVM RBF (ACP)	85,67 %	0,9081	91,4 % ($k = 200$)	Meilleur classifieur classique
ACP + CNN Hybride	94,67 %	0,9919	—	Meilleur modèle global

7.2. Séparation Real/Fake et Cercle des Corrélations

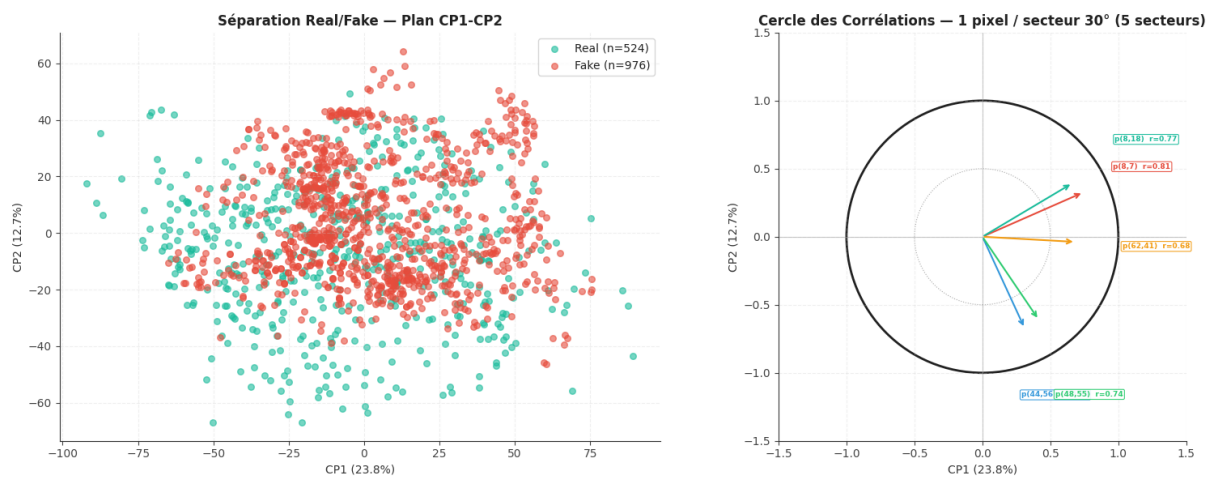


FIGURE 7. Séparation Real/Fake dans le plan CP1-CP2 (gauche) et cercle des corrélations (droite).

Interprétation de la Figure 7 :

Séparation Real/Fake (gauche) :

- Points verts (Real, $n = 524$) et rouges (Fake, $n = 976$) projetés dans le plan CP1-CP2 (23,8 % + 12,7 % de variance).
- Séparation partielle mais significative : les Real tendent vers les valeurs négatives de CP1, les Fake vers les valeurs positives.
- Le chevauchement explique pourquoi les classifieurs linéaires n'atteignent pas 100 %.
- La dispersion plus grande des Real traduit une plus grande variabilité (expressions, éclairages, poses variés).
- Quelques outliers verts éloignés correspondent à des visages atypiques (pose extrême, éclairage inhabituel).

Cercle des Corrélations (droite) :

- Chaque flèche représente un pixel. La longueur indique la corrélation avec CP1/CP2. Les pixels proches du bord sont bien représentés par ces deux composantes.
- $p(8,7)$ avec $r = 0,81$ (rouge) : pixel le plus fortement corrélé à CP1.
- $p(8,18)$ avec $r = 0,77$ (vert) : corrélé aux deux composantes simultanément.
- $p(44,56)$ et $p(48,55)$ avec $r = 0,74$ (bleu) : corrélés négativement à CP2.
- L'angle entre deux flèches indique la corrélation entre pixels : 0 = forte corrélation positive, 90 = indépendance, 180 = anticorrélation.

7.3. Distribution 3D des Individus (CP1–CP2–CP3)

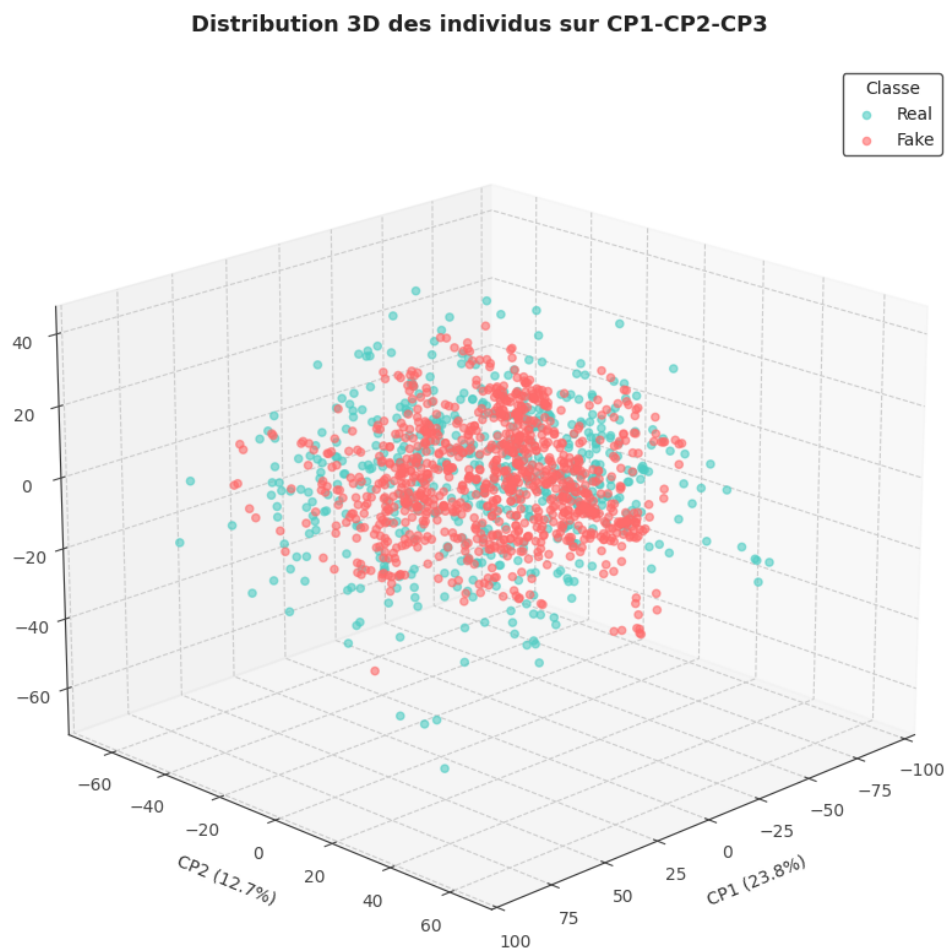


FIGURE 8. Distribution 3D des individus dans l'espace CP1–CP2–CP3 (41,9 % de variance totale).

Interprétation de la Figure 8 :

- CP1 (avant–arrière), CP2 (gauche–droite) et CP3 (bas–haut) représentent collectivement 41,9 % de la variance totale.
- Les images Fake (rouge) forment un nuage dense et compact au centre. Les images Real (vert) sont plus dispersées vers les bords, reflétant leur plus grande variabilité intrinsèque.

- La séparation est visible mais imparfaite : certains points des deux classes se chevauchent, ce qui explique les erreurs résiduelles des classifieurs.
- Les outliers verts aux extrémités correspondent à des visages atypiques (forte expression, éclairage inhabituel).
- La forme ellipsoïdale du nuage est conforme à l'hypothèse gaussienne de l'ACP, validant a posteriori l'utilisation de KNN euclidien.
- Dans la dimension CP3, les Fake tendent vers les valeurs positives et les Real vers les valeurs négatives — information discriminante exploitée par le SVM.

7.4. Sphère des Corrélations 3D

Sphère des Corrélations 3D — 1 pixel / zone sphérique

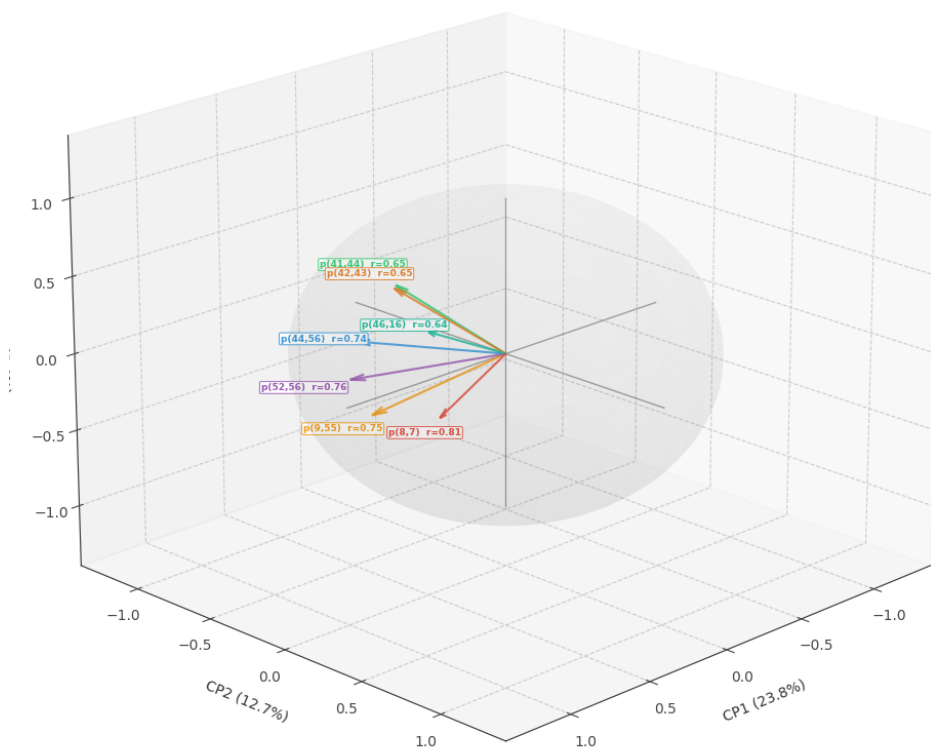


FIGURE 9. Sphère des corrélations 3D : contribution de chaque pixel aux trois premières composantes.

Interprétation de la Figure 9 :

- Extension du cercle des corrélations à trois dimensions. Chaque flèche représente un pixel ; sa position sur la sphère unitaire encode sa corrélation avec CP1, CP2 et CP3.
- p(8,7) avec $r = 0,81$ (rouge) : pixel le mieux représenté ; fortement corrélé à CP1 et CP3, faiblement à CP2.

- $p(41,44)$ et $p(42,43)$ (verts) regroupés dans la même zone : pixels fortement corrélés entre eux, appartenant probablement à la même structure faciale.
- $p(52,56)$ (violet, $r = 0,76$) et $p(9,55)$ (orange, $r = 0,75$) dans des zones distinctes : capturent des types de variation différents.
- La répartition non uniforme révèle des « hotspots » de corrélation correspondant aux régions faciales les plus discriminantes (yeux, nez).

7.5. Matrice de Confusion — Modèle Hybride

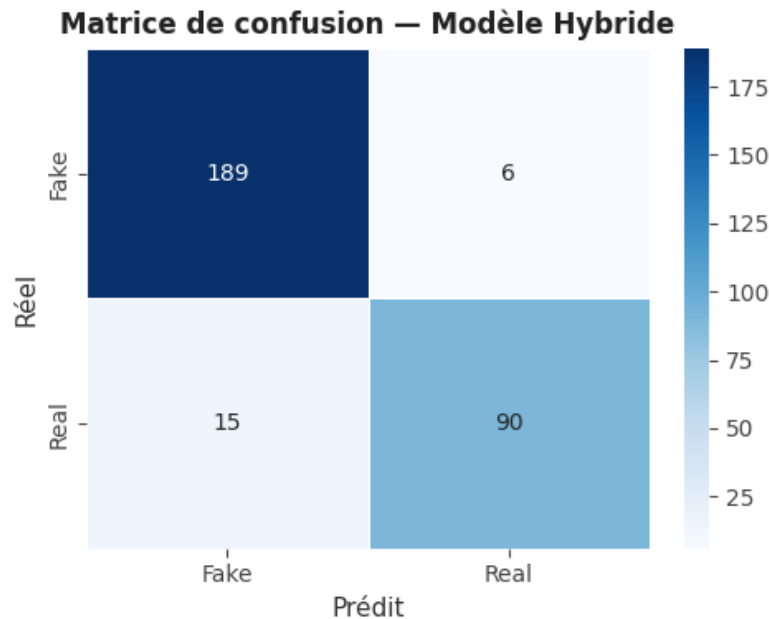


FIGURE 10. Matrice de confusion du modèle hybride ACP+CNN sur les 300 images de test.

Interprétation de la Figure 10 :

- **Vrais Négatifs (TN) = 189** : 189 images Fake correctement classées. Taux = $189/(189 + 6) = 96,9\%$ — très faible taux d'acceptation de falsifications.
- **Vrais Positifs (TP) = 90** : 90 images Real correctement classées. Taux = $90/(90 + 15) = 85,7\%$.
- **Faux Positifs (FP) = 6** : 6 images Fake classées comme Real. Le modèle est très peu enclin à accepter des falsifications — critique pour la sécurité biométrique.
- **Faux Négatifs (FN) = 15** : 15 images Real classées comme Fake. Ces erreurs bloquent un utilisateur légitime — à minimiser pour l'expérience utilisateur.
- Accuracy sur cet échantillon de 300 images = $(189 + 90)/300 = 93,0\%$.
- Précision Real = $90/(90 + 6) = 93,8\%$; Précision Fake = $189/(189 + 15) = 92,6\%$ — valeurs équilibrées malgré le déséquilibre initial des classes.

7.6. Courbe ROC — Modèle Hybride

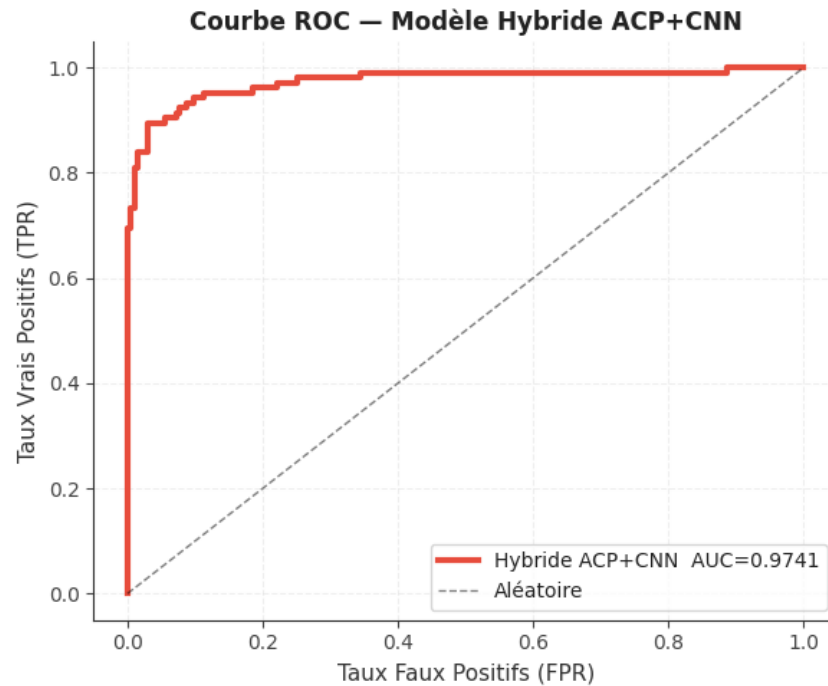


FIGURE 11. Courbe ROC du modèle hybride ACP+CNN ($AUC = 0,9741$).

Interprétation de la Figure 11 :

- $AUC = 0,9741$ — très proche de 1,0. Il y a 97,4 % de chance que le modèle attribue un score plus élevé à une image Real qu'à une image Fake.
- Montée très rapide : à $FPR = 0,05$, $TPR > 0,90$ — on peut détecter 90 % des visages réels avec seulement 5 % de faux positifs.
- Plateau à $TPR \approx 0,97-0,98$, puis convergence vers (1,1) : forme caractéristique d'un excellent classifieur.
- La ligne pointillée grise représente le classifieur aléatoire ($AUC = 0,5$) — notre modèle est très largement supérieur.

7.7. Comparaison des Courbes ROC : SVM vs Hybride

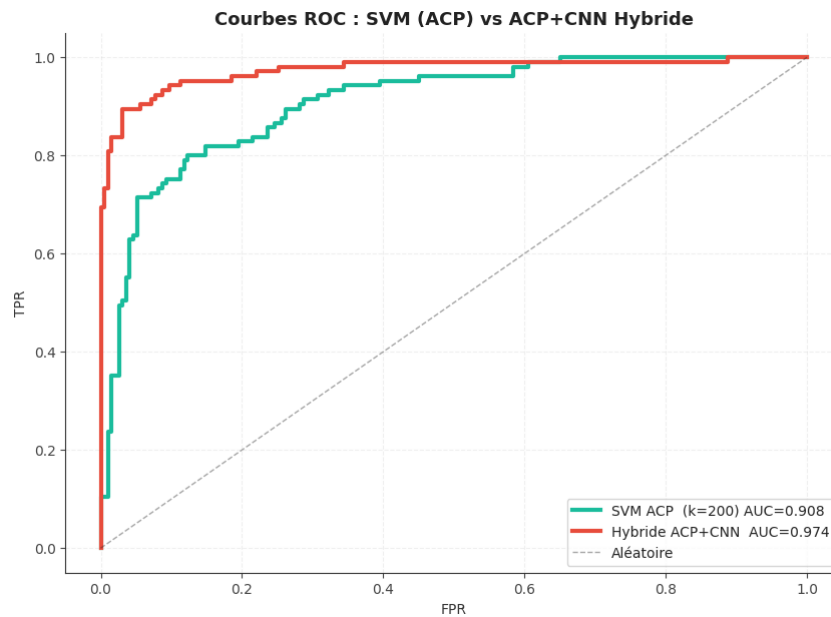


FIGURE 12. Courbes ROC comparatives : SVM avec ACP (AUC=0,908) vs modèle hybride ACP+CNN (AUC=0,974).

Interprétation de la Figure 12 :

- La courbe rouge (Hybride, AUC = 0,974) domine systématiquement la courbe verte (SVM, AUC = 0,908) sur toute la plage de FPR.
- Écart le plus marqué pour les faibles FPR (0–0,2) : à FPR = 0,1, le Hybride atteint TPR $\approx 0,95$ contre TPR $\approx 0,82$ pour le SVM — une différence de 13 points critique pour la sécurité.
- La courbe SVM présente une forme en escalier (scores non calibrés comme probabilités), tandis que la courbe Hybride est régulière (sortie sigmoïde bien calibrée).
- Gain d'AUC de 0,908 à 0,974 = amélioration relative de 7,3 %, avec réduction significative des faux négatifs.
- Ce résultat valide l'hypothèse centrale : la fusion ACP (information globale) + CNN (information locale) capture des caractéristiques complémentaires inaccessibles à chaque approche séparément.

8. Conclusion

8.1. Bilan du Projet

Ce projet a permis de concevoir, implémenter et évaluer un système complet de détection d'anti-spoofing facial. Les principales contributions sont les suivantes :

- **Implémentation manuelle et rigoureuse de l'ACP** en 8 étapes avec NumPy, sans bibliothèque de machine learning pour la réduction de dimension.
- **Validation mathématique** de chaque étape avec formules, code Python et visualisations interprétées.
- **Application au dataset CelebA-Spoof** avec 1 500 images issues de 10 177 sujets distincts.
- **Évaluation comparative** de trois approches : KNN, SVM et modèle hybride ACP+CNN.
- **Résultats solides** : accuracy de 94,67 % et AUC de 0,992 avec le modèle hybride.

8.2. Synthèse des Résultats

KNN (84,33 %) : Résultat satisfaisant pour un classifieur de référence. La distance euclidienne dans l'espace ACP à 200 dimensions est discriminante, mais le classifieur reste sensible au bruit résiduel.

SVM RBF (85,67 %, AUC = 0,908) : Amélioration grâce au noyau RBF qui modélise des frontières non linéaires. Le paramètre `class_weight='balanced'` s'est révélé essentiel pour gérer le déséquilibre des classes.

Hybride ACP+CNN (94,67 %, AUC = 0,992) : Gain de +9 % en accuracy par rapport au SVM. La fusion des features globales (ACP) et locales (CNN) capture des informations complémentaires et conduit aux meilleures performances.

Message Clé du Projet

L'ACP n'est pas seulement un outil de réduction dimensionnelle : c'est une fenêtre sur la structure géométrique des données. Ses fondements mathématiques — matrice de corrélation, décomposition propre, eigenfaces, inertie — permettent de comprendre et d'interpréter les patterns latents dans les images de visages. Cette compréhension profonde est indispensable à tout ingénieur en intelligence artificielle.

Références

1. Yuanhan Zhang et al. (2020). *CelebA-Spoof : Large-Scale Face Anti-Spoofing Dataset with Rich Annotations*. ECCV 2020. <https://github.com/Davidzhangyuanhan/CelebA-Spoof>
2. Jolliffe, I.T. (2002). *Principal Component Analysis*, 2^e édition. Springer.
3. Turk, M. & Pentland, A. (1991). Eigenfaces for Recognition. *Journal of Cognitive Neuroscience*, 3(1), pp. 71–86.
4. Vapnik, V. (1995). *The Nature of Statistical Learning Theory*. Springer.
5. Harris, C.R. et al. (2020). Array programming with NumPy. *Nature*, 585, pp. 357–362.
6. NumPy Documentation — `numpy.linalg.eigh`. <https://numpy.org/doc/stable/reference/generated/numpy.linalg.eigh.html>